

BINUS
UNIVERSITY
INTERNATIONAL

OOP & DS FINAL PROJECT DOCUMENTATION
DIJKSTRA MAZE SOLVER

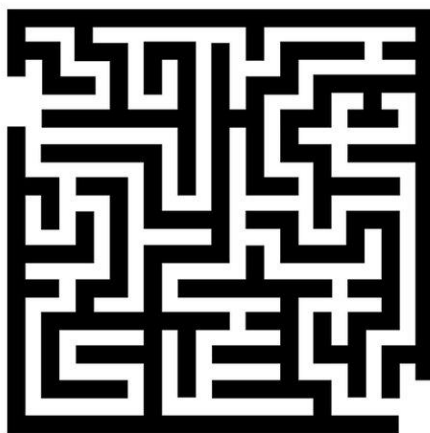
GROUP:

ABDIQANI JIBRIL 2802503665
IMANUEL SHEVA G SIMANJUNTAK 2802499592
PATRICK WILLIAM PRABOWO 2802554520

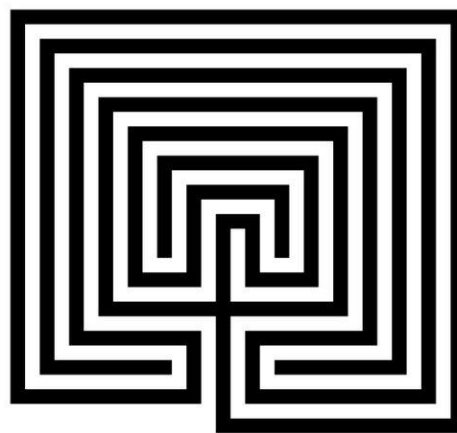
Background

Solving mazes dates back to ancient times with labyrinths. In the early days humans resorted to intuitive methods to solve mazes such as wall following and marking visited paths. With the advancement in the field computer science since the mid 20th century, arises more structured ways to explore paths and find solutions.

We first need to define what exactly a maze is. A maze is a puzzle that contains multiple branches of pathways which is designed to challenge problem-solving skills and logical thinking to find the path from the start to the solution. What makes mazes different from a labyrinth is that labyrinths only has one guided path because labyrinths are more symbolic than a practical puzzle.

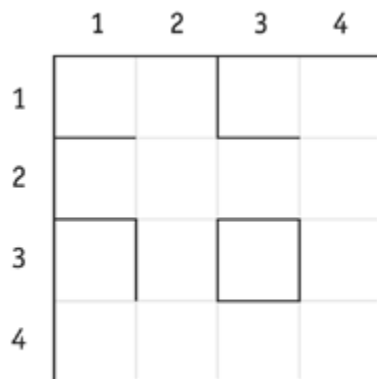


MAZE

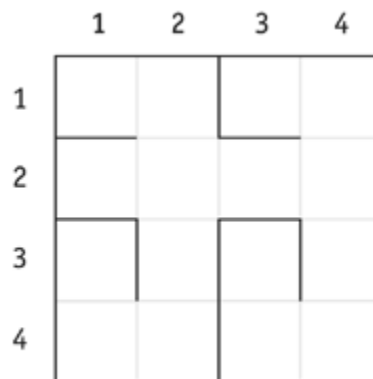


LABYRINTH

There are two types of mazes, a perfect maze and an imperfect maze. A perfect maze is defined as a maze that only has exactly one solution, while an imperfect maze has branches and contains multiple solutions. This means while perfect mazes contain unique paths, imperfect mazes can contain loops or paths that intersect with each other.



IMPERFECT MAZE



PERFECT MAZE

Our maze is an imperfect maze puzzle consisting of a grid with paths and walls, where the goal is to navigate from a start point to an endpoint. In this project, we focus on 3 different maze sizes: 10x10, 15x15, and 20x20, manageable yet complex sizes that allow for pathfinding challenges.

Problem Description

Since imperfect mazes have many different solutions, using conventional algorithms such as BFS would not always yield the most efficient solution. Therefore we need an algorithm with a heuristic function to find the most efficient solution amongst others.

This is where Dijkstra's Shortest Path algorithm becomes useful. Dijkstra was invented by a Dutch computer scientist named Edsger W. Dijkstra in 1956. Dijkstra's algorithm is a greedy algorithm used to find the shortest path from a single source node to all other nodes in a weighted graph with non-negative edge weights. It works by iteratively selecting the closest unvisited node and updating the distances to its neighbors.

The challenge with implementing Dijkstra's algorithm is finding the best data structure for storing and sorting the nodes in a minimum cost priority which is needed for the solver to access during the process.

The other challenge is how to store the maze data in such a way that it allows the solver to traverse efficiently during the solving process and without using too much memory. From these tests we can find the best combination of data structures to use for this problem.

Solution

The Maze

Our maze will be made in the form of a matrix which consists of either a wall or a path in each cell. The user is able to create their maze (set walls and remove walls wherever they like) and the program will show the shortest possible solution or show that there is no solution (if there is none).

2D Array vs Hashmap

In Java, a 2D array is a built-in data structure that stores data in a grid-like format using rows and columns, allowing direct access to any element through integer-based indexing with constant time complexity. We chose to implement a 2D array because it provides a simple and efficient way to represent structured layouts like seating charts, game boards, or mazes where each position can be directly addressed with predictable coordinates. On the other hand, a HashMap is also backed by arrays internally but stores key-value pairs using a hashing function, which involves additional steps such as computing hash codes and handling collisions through buckets or chaining. Although this allows for more flexible key types and dynamic resizing, it adds overhead compared to the straightforward access in a 2D array. HashMaps excel when data needs to be quickly looked up using non-integer keys or when the structure is

sparse and doesn't follow a fixed grid. In our case, we're evaluating whether this extra layer of hashing and abstraction is justified, considering that a 2D array already provides fast direct access for our needs. If the data model is simple and fixed in structure, 2D arrays are more memory-efficient and faster for index-based operations. However, if flexibility in indexing or expansion is required like using names, coordinates, or IDs as keys then the benefits of HashMap could outweigh the cost of added complexity.

The Solver

Dijkstra's Algorithm pseudocode:

```
// Step 1: Initialization
create distance map dist, set all nodes to INFINITY
dist[source] = 0

create a min-priority queue Q
add all nodes to Q with their dist values

// Step 2: Main Loop
while Q is not empty:
    u = node in Q with smallest dist[u]
    remove u from Q

    for each neighbor v of u:
        alt = dist[u] + weight(u, v)
        if alt < dist[v]:
            dist[v] = alt
            update priority of v in Q to dist[v]

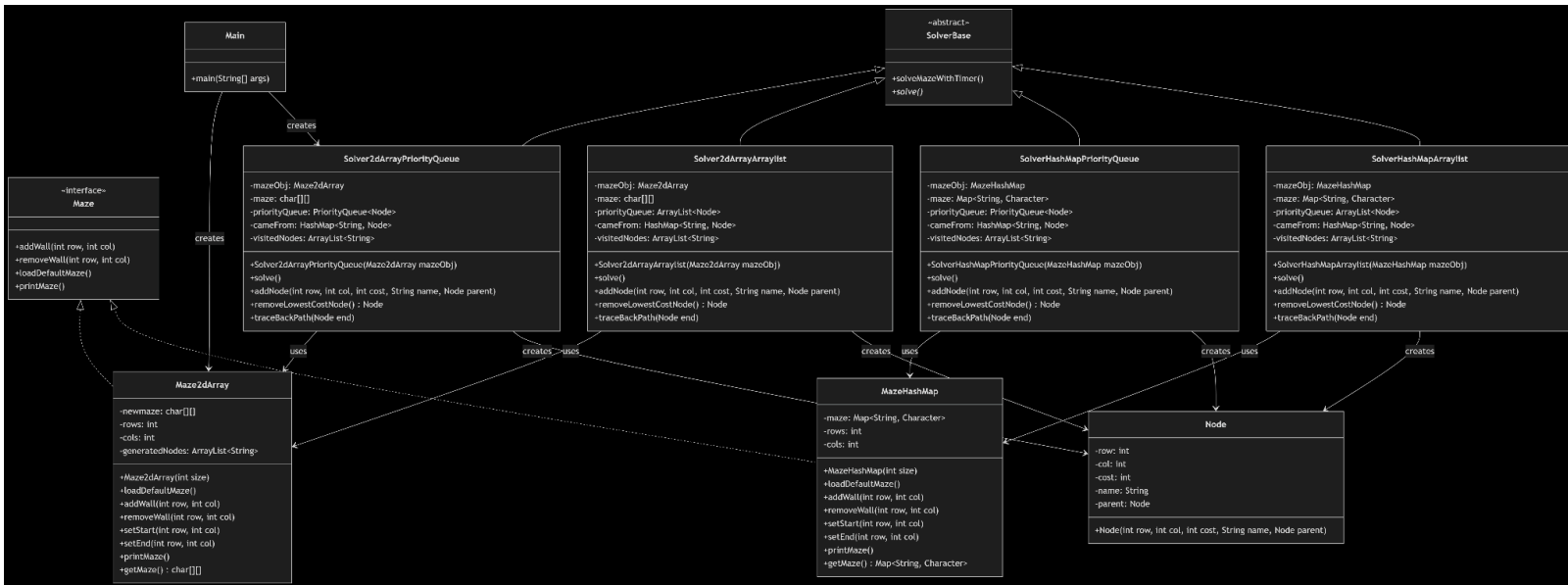
return dist
```

ArrayList vs Priority Queue

An ArrayList is a resizable array that allows indexed access and dynamic storage of elements, making it simple to use when all elements need to be iterated over or modified frequently. It is part of the Java Collections Framework and supports random access through the RandomAccess interface, allowing constant-time lookup by index. However, for Dijkstra's algorithm in a maze solver, where the next node with the lowest cost must be selected at each step, using an ArrayList becomes inefficient because it requires scanning the entire list to find the minimum cost node, resulting in $O(n)$ time for each extraction. In contrast, a PriorityQueue is specifically designed to efficiently retrieve the minimum (or maximum) element based on a priority defined by a comparator or natural ordering. Internally, it uses a binary heap structure which keeps the smallest element at the top, allowing polling the minimum element in $O(\log n)$ time.

Code Implementation

UML diagram



How the algorithm works:

1. Initialize.

```

public void solve() { 4 usages
    currentRow = 0;
    currentColumn = 0;

    Node startNode = new Node(currentRow, currentColumn, cost: 0, name: "S", parent: null);
    priorityQueue.add(startNode);
    cameFrom.put("S", null);
    visitedNodes.add("S");
}
    
```

- The start node is always set at row 0 column 0.
- Creates a Node object with the currentRow: 0, currentColumn: 0, name: 'S', and null as parent.
- Adds it to the queue

2. Main loop: search for path

```
while (!priorityQueue.isEmpty()) {
    // Step 1: get and remove the lowest-cost node
    Node current = removeLowestCostNode();
}
```

- From the current set of nodes in the priority queue, it chooses the lowest cost one.
- For the arraylist version, the removeLowestCostNode function has to iterate through every element in the arraylist, in order to find the lowest cost node:

```
public Node removeLowestCostNode() { 1 usage
    if (priorityQueue.isEmpty()) return null;

    Node lowest = priorityQueue.get(0);
    int lowestIndex = 0;

    for (int i = 1; i < priorityQueue.size(); i++) {
        Node current = priorityQueue.get(i);
        if (current.cost < lowest.cost) {
            lowest = current;
            lowestIndex = i;
        }
    }

    return priorityQueue.remove(lowestIndex);
}
```

- Whereas in the actual priorityqueue implementation, the method is much simpler:

```
public Node removeLowestCostNode() { 1 usage
    return priorityQueue.poll(); // Automatically retrieves lowest-cost node
}
```

```
// Step 2: if it's the end, trace the path
if (maze[current.row][current.col] == 'E') {
    System.out.println("Reached the End!");
    traceBackPath(current);
    return;
}
```

- It also checks if it reaches the end, 'E', if so it will trace back the path from E to S, if not, continue.

```
// Step 3: expand the node in all directions
for (int[] dir : directions) {
    int newRow = current.row + dir[0];
    int newCol = current.col + dir[1];

    if (isValid(newRow, newCol) && (maze[newRow][newCol] == '0' || maze[newRow][newCol] == 'E')) {
        String nodeName = getNodeName(newRow, newCol);
        int newCost = current.cost + 1;
        addNode(newRow, newCol, newCost, nodeName, current);
    }
}
```

- It then expands the node in 4 directions while also checking if the move is valid (within bounds and not a wall)
- If the direction is valid, it will increase the cost by 1 and add the new node into the queue with information such as coordinates, cost, name, and parent.)

```
public void addNode(int row, int col, int cost, String name, Node parent) { 1 usage
    if (visitedNodes.contains(name)) return;

    Node newNode = new Node(row, col, cost, name, parent);
    priorityQueue.add(newNode);
    cameFrom.put(name, parent);

    visitedNodes.add(name);
}
```

- When addNode is called, the name of the current node and the name of its parent is added to the cameFrom hashmap. It looks something like this {nodeName : parentName} Here the name is key and parent is the value.

```
while (!priorityQueue.isEmpty()) {
    Node current = removeLowestCostNode();
```

- Once it has explored all 4 directions and none of them are the end point, as long as the priority queue is not empty, the current node will be set to the node with the lowest cost, while simultaneously removing that node from the queue.
- Thus, from that new current node it will explore all 4 directions again.

3. Traceback when we have reached the end.

```
public void traceBackPath(Node end) { 1 usage
    LinkedList<String> finalPath = new LinkedList<>();
    Node current = end;

    while (current != null) {
        finalPath.addFirst(current.name);
        current = cameFrom.get(current.name);

        if (current != null && !current.name.equals("S")){
            maze[current.row][current.col] = '.';
        }
    }
}
```

- Once the end is reached, a linked list is created and the node names are added from the cameFrom hashmap by accessing the value, which is the name of the parent node.
- This way, only the correct and shortest path is added to the finalPath.

Hypothesis

We expect that the maze stored as a 2D array and using priority queue solver would have the best overall solving time. This is because:

- A 2D array provides direct index-based access $O(1)$ which has low overhead, whereas the hashmap would need another function for parsing and hashing, which could make for some overhead.
- The priority queue is able to do efficient minimum cost retrieval, whereas the arraylist needs another function to scan the list for the lowest cost element and pop it out.

Testing Result

Maze Solver Timing Table (ms)

Maze Size: 10x10

Attempts	2DArray Maze + ArrayList Solver	2DArray Maze + PriorityQueue Solver	HashMap Maze + ArrayList	HashMap Maze + PriorityQueue Solver
1	0.8	1.02	1.42	1.19
2	0.95	1.08	0.81	1.14
3	0.9	0.83	1.03	1.06
4	1.1	0.95	1.44	0.91
5	1.46	0.94	1.42	1.25
6	1.54	0.98	0.88	0.92
7	1.16	0.84	0.88	0.95
8	0.91	1.06	1.2	1.12
9	0.95	0.94	1.18	1.1
10	1.07	0.98	0.89	0.94
Average	1.084	0.962	1.115	1.0580

Maze Size: 15x15

Attempts	2DArray Maze + ArrayList Solver	2DArray Maze + PriorityQueue Solver	HashMap Maze + ArrayList	HashMap Maze + PriorityQueue Solver
1	1.2	1.32	2.35	1.96
2	1.35	1.28	2.21	1.9
3	1.48	1.41	2.14	1.86
4	1.42	1.45	1.95	2.16
5	1.28	1.25	2.16	2.12
6	1.18	1.19	3.51	2.88
7	1.45	1.47	2.14	3.47
8	1.32	1.38	2.07	3.16
9	1.38	1.35	2.43	2.95
10	1.41	1.29	1.86	2.75
Average	1.347	1.339	2.282	2.521

Maze size: 20x20

Attempts	2DArray Maze + ArrayList Solver	2DArray Maze + PriorityQueue Solver	HashMap Maze + ArrayList	HashMap Maze + PriorityQueue Solver
1	1.12	1.41	3.32	3.4
2	1.54	1.54	4.02	2.22
3	2.23	1.65	2.26	2.58
4	1.61	1.5	2.81	3.86
5	1.53	1.46	2.88	3.09
6	1.39	1.25	2	3.41
7	1.27	1.77	2.46	2.64
8	2.05	1.6	3.11	1.87
9	1.44	1.43	2.8	3.12
10	1.41	1.48	3.88	2.55
Average	1.559	1.509	2.954	2.874

Memory Usage for Operations: addWall(), removeWall() and Solve() (kilobytes)

Size: 10x10

Attempts	2DArray Maze + ArrayList Solver	2DArray Maze + PriorityQueue Solver	HashMap Maze + ArrayList	HashMap Maze + PriorityQueue Solver
1	1966	1966	2457	2457
2	2457	2457	2949	1966
3	2457	2457	1966	1966
4	1966	2457	2457	2457
5	1966	2457	2949	2949
6	2457	2949	1966	1966
7	2949	2457	2457	2949
8	1966	1966	1966	2457
9	2457	1966	2949	2457
10	2457	2457	1966	2949
Average	2309	2359	2308	2358

Size: 15x15

Attempts	2DArray Maze + ArrayList Solver	2DArray Maze + PriorityQueue Solver	HashMap Maze + ArrayList	HashMap Maze + PriorityQueue Solver
1	1966	2457	2949	2457
2	2007	2457	2457	2949
3	2457	1966	2949	3439
4	2949	2457	1966	2949
5	1966	1966	2457	1966

6	2457	2457	2457	2949
7	2457	2457	2457	2949
8	2457	2457	2457	2457
9	2457	2457	2949	2949
10	2007	2457	3439	2457
Average	2313	2357	2566	2696

Size: 20x20

Attempts	2DArray Maze + ArrayList Solver	2DArray Maze + PriorityQueue Solver	HashMap Maze + ArrayList	HashMap Maze + PriorityQueue Solver
1	2949	2457	3439	2949
2	2457	1966	3439	3439
3	2457	2457	3439	3439
4	2457	1966	2950	2457
5	2457	2457	2457	1474
6	1966	2457	2457	2457
7	2457	1966	2950	2457
8	2457	2457	2950	2949
9	2457	2457	2457	2949
10	1966	2457	1966	3439
Average	2418	2315	2754	2797

Add Wall and Delete Wall Timing Tables(ms)

For each of these tests, the operations are done to the middle of the maze

Size 10x10

Method	Add Wall	Add Wall	Remove Wall	Remove Wall
Structure	2DArray	HashMap	2DArray	HashMap
Test 1	0.0010	0.0018	0.0012	0.0042
Test 2	0.0011	0.0020	0.0011	0.0038
Test 3	0.0009	0.0021	0.0013	0.0036
Test 4	0.0012	0.0017	0.0012	0.0041
Test 5	0.0011	0.0018	0.0011	0.0034
Test 6	0.0009	0.0019	0.0010	0.0033
Test 7	0.0010	0.0016	0.0011	0.0039
Test 8	0.0011	0.0018	0.0012	0.0040
Test 9	0.0010	0.0019	0.0010	0.0043
Test 10	0.0011	0.0017	0.0011	0.0041
Average	0.0010	0.0018	0.0011	0.0039

Size 15x15

Method	Add Wall	Add Wall	Remove Wall	Remove Wall
Structure	2DArray	HashMap	2DArray	HashMap
Test 1	0.0014	0.0025	0.0015	0.0051
Test 2	0.0015	0.0028	0.0016	0.0048
Test 3	0.0013	0.0031	0.0017	0.0042
Test 4	0.0016	0.0027	0.0018	0.0059
Test 5	0.0018	0.0026	0.0014	0.0036

Test 6	0.0013	0.0029	0.0013	0.0041
Test 7	0.0014	0.0023	0.0015	0.0046
Test 8	0.0015	0.0026	0.0018	0.0049
Test 9	0.0014	0.0025	0.0014	0.0053
Test 10	0.0015	0.0024	0.0016	0.0047
Average	0.0015	0.0026	0.0016	0.0047

Size 20x20

Method	Add Wall	Add Wall	Remove Wall	Remove Wall
Structure	2DArray	HashMap	2DArray	HashMap
Test 1	0.0016	0.0029	0.0018	0.0076
Test 2	0.0018	0.0033	0.0024	0.0065
Test 3	0.0019	0.0056	0.0025	0.0044
Test 4	0.0021	0.0028	0.0025	0.0125
Test 5	0.0049	0.003	0.0021	0.0035
Test 6	0.002	0.0028	0.0019	0.0049
Test 7	0.0023	0.0023	0.0066	0.0066
Test 8	0.0019	0.003	0.0025	0.0063
Test 9	0.0017	0.003	0.0017	0.0139
Test 10	0.0017	0.0029	0.002	0.147
Average	0.0022	0.0032	0.0026	0.0081

Analysis

Time complexity:

Operation	2D Array (arr[i][j])	HashMap (map.get(key))
Access	O(1)	O(1) average, O(n) worst
Insert	O(1)	O(1) average, O(n) worst
Delete	O(1)	O(1) average, O(n) worst

2DArray vs HashMap

The 2DArray provides constant time for storing and accessing maze data. For accessing or modifying a cell, it takes O(1) time as it uses direct index-based access. Thus, it is very efficient for operations like checking neighbours, and adding or removing walls. On the other hand, the HashMap implementation uses hashing for key-based access which has an average of O(1) time complexity, but due to the use of string keys and potential collisions, the HashMap is slower than the 2dArray.

ArrayList vs PriorityQueue

The ArrayList implementation has O(1) time complexity inserting elements, but when it comes to retrieving the lowest cost node, it first has to scan the entire list, thus it has O(n) time per selection: as the number of nodes increase, the linear search becomes more inefficient. On the other hand, the PriorityQueue implementation maintains a min heap. This allows more efficient retrieval of the lowest cost node in O(log n) time. No additional scanning is required as the PriorityQueue already sorts the elements internally. This improves the performance greatly, especially in larger maze sizes. Overall, the combination of 2DArray and PriorityQueue has the fastest solving time due to fast access and lowest cost node retrieval while the HashMap and ArrayList combination is the slowest.

Memory usage:

Feature	2D Array	HashMap
Structure	Array of arrays	Array of buckets + node chains
Memory per element	~4 bytes (int)	~64–100 bytes per entry

2DArray vs HashMap

The 2D array (`char[][]`) generally uses less memory than the Hashmap version. This is because each element directly represents a cell in the maze thus it has minimal overhead. Additionally, access to any cell is much more simple than the Hashmap implementations: it uses simple row and column indexing to access any cell. On the other hand, the Hashmap implementation stores keys as stringified coordinates such as "2,5". This adds some overhead due to the storage of key objects and hash codes. Thus, hashmaps are more flexible but less memory-efficient, especially in larger mazes.

Arraylist vs PriorityQueue

As for the two solvers, Arraylist uses slightly less memory than PriorityQueue because an Arraylist does not maintain any special structure other than storing elements: It is just an array of node objects. On the other hand, a PriorityQueue has to maintain a binary heap which is ordered by the elements' cost. This creates overhead, which may not be too significant, but it makes the PriorityQueue implementation require a little more memory usage. After testing, the difference in memory usage between the ArrayList and the PriorityQueue is very minimal; often only a few kilobytes. Overall, the most memory efficient combination was 2DArray with ArrayList, while the worst memory usage was Hshmap with PriorityQueue.

Conclusion

Based on the results, the best data structure combination is 2DArray with PriorityQueue in terms of both speed and memory usage. The priority queue had more efficient node selection for pathfinding due to its min heap structure, while the 2d array allowed for fast and direct access to maze cells. Although the 2d array + Arraylist implementation had slightly lower memory usage, the difference is negligible and the performance trade-off in solving time is not worth the minimal difference in memory usage. As for the HashMap implementations, they were a bit slower and required more memory usage. This is because it uses a hashing function to store key-value pairs, and it potentially handles collisions which makes for some overhead.

Appendix

Github link: <https://github.com/Sheva123456/Maze-Solver-/tree/solver-full>

Presentation link: <https://www.canva.com/design/DAGqWhqPfZ0/sQcHZWTrHGHbHStuq5vBYg/edit>

How to use:

- In Main, there are 2 ‘warm up’ runs before actually calculating the time taken to execute solve(). This is because Java uses a JIT compiler, which optimizes code only after it's run a few times. Initial runs are slower, so warm-up executions are done first to ensure accurate performance measurements after JVM optimization.

```
// --- Warmup 1 ---
Maze2dArray maze = new Maze2dArray( size: 10);
System.out.println("Warm up 1:");
SolverBase solver1 = new Solver2dArrayArrayList(maze);
solver1.solve();
System.out.println("");

// --- Warmup 2 :
System.out.println("Warm up 2:");
Maze2dArray maze2 = new Maze2dArray( size: 10);
maze2.loadDefaultMaze();
SolverBase solver2 = new Solver2dArrayArrayList(maze2);
solver2.solve();
System.out.println("");
```

- In order to run the program, the user must create a new maze object based on their desired maze data structure (either Maze2dArray or MazeHashMap), while also inputting the desired size (10, 15 or 20)

```
Maze2dArray maze = new Maze2dArray( size: 10);
```

```
MazeHashMap maze = new MazeHashMap( size: 10);
```

- The user can either create a custom maze from zero by using addWall(row,col) and removeWall(row,col) or load a prebuilt maze based on the size by calling loadDefaultMaze() (addwall and removewall can still be used to modify the default maze)

```
maze3.addWall( row: 5, col: 5);
```


Priority Q Hashmap maze:

Reached the End!

Final path: [S, q01, q11, q12, q13, q14, q15, q16, q17, q27, q37, q36, q35, q34, q33, q32

```
S . 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 . . . . . . 1 0 0 0 0 0 1
1 1 1 1 1 1 1 . 1 0 1 1 1 0 1
1 . . . . . . 1 0 0 0 1 0 1
1 . 1 1 1 1 1 1 1 1 1 0 1 0 1
1 . 0 0 0 0 0 0 0 0 0 1 0 1 0 1
1 . 1 1 1 1 1 1 1 0 1 . . . 1
1 . . . . . . 1 0 1 . 1 . 1
1 0 1 1 1 1 1 . 1 0 1 . 1 . 1
1 0 0 0 0 0 1 . 1 0 1 . 1 . 1
1 1 1 1 1 0 1 . 1 0 1 . 1 . 1
1 0 0 0 0 0 1 . 1 0 1 . 1 . 1
1 0 1 1 1 1 1 . . . 1 . 1 . 1
1 0 0 0 0 0 0 0 1 . . . 1 . 1
1 1 1 1 1 1 1 1 1 1 1 1 . E
```

Maze solved in 2.16 milliseconds.

References

- Buck, J. (n.d.). *Mazes for Programmers: Code Your Own Twisty Little Passages*. The Pragmatic Programmers LLC.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3rd ed.). MIT Press.
- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269–271. <https://doi.org/10.1007/BF01386390>
- Dijkstra, E. W. (1976). *A discipline of programming*. Prentice Hall.
- GeeksforGeeks. (2025, April 9). What is Dijkstra's Algorithm? | *Introduction to Dijkstra's Shortest Path Algorithm*. GeeksforGeeks.
<https://www.geeksforgeeks.org/introduction-to-dijkstras-shortest-path-algorithm/>
- Javaid, A. (2013). Understanding Dijkstra Algorithm. *SSRN Electronic Journal*.
<https://doi.org/10.2139/ssrn.2340905>
- Koenig, S., & Likhachev, M. (2002). D* Lite. In *Proceedings of the AAAI Conference on Artificial Intelligence* (Vol. 15, pp. 476–483). AAAI Press.
<https://www.aaai.org/Papers/AAAI/2002/AAAI02-071.pdf>
- Koffman, E. B., & Wolfgang, P. A. (2021). *Data structures: abstraction and design using Java*. -: John Wiley & Sons.
- Weiss, M. A (2010). *Data Structures and Problem Solving Using Java (Fourth Edition)*. -: Addison-Wesley.
- Yap, P. H. (2002). Grid-based path-finding. In M. Bubak, J. Dongarra, A. Hoisie, J. Wasniewski, & J. C. T. Yang (Eds.), *Computational Science — ICCS 2002. Lecture Notes in Computer Science* (Vol. 2338, pp. 44–55). Springer. https://doi.org/10.1007/3-540-47789-6_5